

```

import os
import re
import logging
import tempfile
import subprocess
import pandas as pd
from collections import defaultdict
from openpyxl import Workbook
from openpyxl.styles import Font, PatternFill, Alignment, Border, Side
from openpyxl.utils import get_column_letter
from telegram import Update
from telegram.ext import ApplicationBuilder, MessageHandler, CommandHandler, filt

TOKEN = os.environ.get("BOT_TOKEN", "YOUR_TOKEN_HERE")
logging.basicConfig(level=logging.INFO)

def parse_xlsb(filepath):
    out_dir = tempfile.mkdtemp()
    subprocess.run(
        ["libreoffice", "--headless", "--convert-to", "xlsx", "--outdir", out_dir,
        capture_output=True, text=True, timeout=30
    )
    xlsx_files = [f for f in os.listdir(out_dir) if f.endswith(".xlsx")]
    if not xlsx_files:
        raise Exception("Konvertatsiya muvaffaqiyatsiz")
    return pd.read_excel(os.path.join(out_dir, xlsx_files[0]), sheet_name=0, head

def clean_name(s):
    s = str(s)
    parts = s.split("/")
    name = parts[-1].strip() if len(parts) >= 3 else s.strip()
    name = re.sub(r'^00634', '', name).strip()
    if "SmartVista" in name or "LOGISTICS TRA" in name:
        return "SmartVista (Terminal)"
    if len(name) > 40:
        name = name[:40] + "..."
    return name if name else "Noma'lum"

def extract_transactions(df):
    header_row_idx = None
    for i, r in df.iterrows():
        if str(r.iloc[0]).strip() == "Data" or str(r.iloc[0]).strip() == "\u0414\
        header_row_idx = i
        break

```

```

if header_row_idx is None:
    return [], 0, 0
txs = []
total_debit = 0
total_credit = 0
for i in range(header_row_idx + 1, len(df)):
    r = df.iloc[i]
    date_val = r.iloc[0]
    if pd.isna(date_val):
        continue
    date_str = str(date_val)
    if "\u0418\u0442\u043e\u0433" in date_str:
        try:
            total_debit = float(r.iloc[5]) if pd.notna(r.iloc[5]) else 0
            total_credit = float(r.iloc[6]) if pd.notna(r.iloc[6]) else 0
        except:
            pass
        continue
    try:
        date_fmt = date_val.strftime("%d.%m.%Y") if hasattr(date_val, 'strftime')
        time_fmt = date_val.strftime("%H:%M") if hasattr(date_val, 'strftime')
        doc_no = str(int(r.iloc[2])) if pd.notna(r.iloc[2]) else ""
        account_raw = str(r.iloc[1]) if pd.notna(r.iloc[1]) else ""
        counterparty = clean_name(account_raw)
        debit = float(r.iloc[5]) if pd.notna(r.iloc[5]) else None
        credit = float(r.iloc[6]) if pd.notna(r.iloc[6]) else None
        desc = str(r.iloc[7]).strip() if pd.notna(r.iloc[7]) else ""
        if desc.startswith("00634"):
            desc = desc[5:].strip()
        txs.append({
            "date": date_fmt, "time": time_fmt, "doc": doc_no,
            "counterparty": counterparty, "debit": debit, "credit": credit, "
        })
    except:
        continue
return txs, total_debit, total_credit

def build_summary(txs):
    debit_by = defaultdict(lambda: {"sum": 0, "count": 0})
    credit_by = defaultdict(lambda: {"sum": 0, "count": 0})
    for tx in txs:
        name = tx["counterparty"]
        if tx["debit"]:
            debit_by[name]["sum"] += tx["debit"]
            debit_by[name]["count"] += 1
        if tx["credit"]:
            credit_by[name]["sum"] += tx["credit"]

```

```

        credit_by[name]["count"] += 1
    return debit_by, credit_by

def thin_border():
    s = Side(style="thin", color="DDDDDD")
    return Border(left=s, right=s, top=s, bottom=s)

def sty(cell, bold=False, bg=None, color="000000", size=10, align="left"):
    cell.font = Font(bold=bold, color=color, size=size, name="Arial")
    if bg:
        cell.fill = PatternFill("solid", fgColor=bg)
    cell.alignment = Alignment(horizontal=align, vertical="center", wrap_text=True)
    cell.border = thin_border()

def create_excel(df, output_path):
    wb = Workbook()
    ws1 = wb.active
    ws1.title = "Kochirma"

    DARK = "1B3A6B"
    LBLUE = "D6E4F7"
    RED = "C0392B"
    GREEN = "1E8449"
    WHITE = "FFFFFF"
    YELLOW = "FFF3CD"

    bank = str(df.iloc[0, 0]).strip()
    period = str(df.iloc[1, 0]).strip()
    account_raw = str(df.iloc[2, 0]).replace('\xa0', ' ').strip()
    owner = str(df.iloc[3, 7]).strip() if pd.notna(df.iloc[3, 7]) else ""

    def extract_num(s):
        nums = re.findall(r'\d[\d\s]*\.\d*', str(s).replace('\xa0', ' ').replace(
            for n in nums:
                try:
                    return float(n)
                except:
                    continue
        return 0

    bal_start = extract_num(str(df.iloc[3, 0]))
    bal_end = extract_num(str(df.iloc[3, 1]))

    row = 1
    ws1.merge_cells("A%d:H%d" % (row, row))
    c = ws1.cell(row=row, column=1, value=bank)
    sty(c, bold=True, bg=DARK, color=WHITE, size=12, align="center")

```

```

ws1.row_dimensions[row].height = 26
row += 1

ws1.merge_cells("A%d:H%d" % (row, row))
c = ws1.cell(row=row, column=1, value=period)
sty(c, bg=LBLUE, size=10, align="center")
ws1.row_dimensions[row].height = 20
row += 1

ws1.merge_cells("A%d:H%d" % (row, row))
c = ws1.cell(row=row, column=1, value=account_raw)
sty(c, size=9, align="center")
ws1.row_dimensions[row].height = 18
row += 1

if owner and owner != "nan":
    ws1.merge_cells("A%d:H%d" % (row, row))
    c = ws1.cell(row=row, column=1, value="Egasi: " + owner)
    sty(c, bold=True, bg=YELLOW, size=10, align="center")
    ws1.row_dimensions[row].height = 20
    row += 1

row += 1

for i, (lbl, val, col) in enumerate([
    ("Boshlanish goldighi", bal_start, DARK),
    ("Tugash goldighi", bal_end, RED if bal_end < bal_start else GREEN)
]):
    c1 = get_column_letter(1 + i * 4)
    c2 = get_column_letter(3 + i * 4)
    ws1.merge_cells("%s%d:%s%d" % (c1, row, c2, row))
    c = ws1.cell(row=row, column=1 + i * 4, value=lbl)
    sty(c, bold=True, bg=col, color=WHITE, align="center")
    ws1.row_dimensions[row].height = 20
    ws1.merge_cells("%s%d:%s%d" % (c1, row + 1, c2, row + 1))
    c2cell = ws1.cell(row=row + 1, column=1 + i * 4, value=val)
    sty(c2cell, bold=True, bg=LBLUE, align="center", size=13)
    c2cell.number_format = '#,##0.00'
    ws1.row_dimensions[row + 1].height = 24
row += 3

headers = ["Sana", "Vaqt", "Hujjat", "Kontragent", "Chiqim", "Kirim", "Izoh"]
for col_i, h in enumerate(headers, 1):
    c = ws1.cell(row=row, column=col_i, value=h)
    sty(c, bold=True, bg=DARK, color=WHITE, align="center")
ws1.row_dimensions[row].height = 20
row += 1

```

```

txs, total_debit, total_credit = extract_transactions(df)
for tx in txs:
    bg = "FFE8E8" if tx["debit"] else "E8FFE8"
    vals = [tx["date"], tx["time"], tx["doc"], tx["counterparty"], tx["debit"]
    for col_i, val in enumerate(vals, 1):
        c = ws1.cell(row=row, column=col_i, value=val)
        sty(c, bg=bg, size=9)
        if col_i == 5 and val:
            sty(c, bold=True, bg=bg, color=RED, align="right", size=10)
            c.number_format = '#,##0.00'
        if col_i == 6 and val:
            sty(c, bold=True, bg=bg, color=GREEN, align="right", size=10)
            c.number_format = '#,##0.00'
    ws1.row_dimensions[row].height = 16
    row += 1

row += 1
for lbl, val, col, col_i in [
    ("Jami chiqim:", total_debit, RED, 5),
    ("Jami kirim:", total_credit, GREEN, 6)
]:
    ws1.merge_cells("A%d:D%d" % (row, row))
    c = ws1.cell(row=row, column=1, value=lbl)
    sty(c, bold=True, align="right", size=11)
    c2 = ws1.cell(row=row, column=col_i, value=val)
    sty(c2, bold=True, color=col, align="right", size=12)
    c2.number_format = '#,##0.00'
    ws1.row_dimensions[row].height = 22
    row += 1

widths1 = [12, 7, 13, 28, 16, 16, 40]
for i, w in enumerate(widths1, 1):
    ws1.column_dimensions[get_column_letter(i)].width = w

ws2 = wb.create_sheet("Firmalar")
debit_by, credit_by = build_summary(txs)
all_names = set(list(debit_by.keys()) + list(credit_by.keys()))

row2 = 1
ws2.merge_cells("A%d:E%d" % (row2, row2))
c = ws2.cell(row=row2, column=1, value="FIRMALAR BO'YICHA JAMLAMA")
sty(c, bold=True, bg=DARK, color=WHITE, size=13, align="center")
ws2.row_dimensions[row2].height = 28
row2 += 1

ws2.merge_cells("A%d:E%d" % (row2, row2))

```

```

c = ws2.cell(row=row2, column=1, value=period)
sty(c, bg=LBLUE, size=10, align="center")
ws2.row_dimensions[row2].height = 20
row2 += 2

for col_i, h in enumerate(["Firma nomi", "Chiqim soni", "Jami chiqim", "Kirim
    c = ws2.cell(row=row2, column=col_i, value=h)
    sty(c, bold=True, bg=DARK, color=WHITE, align="center")
ws2.row_dimensions[row2].height = 22
row2 += 1

for name in sorted(all_names):
    d = debit_by.get(name, {"sum": 0, "count": 0})
    cr = credit_by.get(name, {"sum": 0, "count": 0})
    bg = "FFF0F0" if d["sum"] > cr["sum"] else "F0FFF0" if cr["sum"] > d["sum"]
    row_data = [name, d["count"] or "", d["sum"] or "", cr["count"] or "", cr["sum"] or ""]
    for col_i, val in enumerate(row_data, 1):
        c = ws2.cell(row=row2, column=col_i, value=val)
        sty(c, bg=bg, size=10)
        if col_i == 3 and val:
            sty(c, bold=True, bg=bg, color=RED, align="right", size=11)
            c.number_format = '#,##0.00'
        if col_i == 5 and val:
            sty(c, bold=True, bg=bg, color=GREEN, align="right", size=11)
            c.number_format = '#,##0.00'
    ws2.row_dimensions[row2].height = 20
    row2 += 1

for col, w in zip(["A", "B", "C", "D", "E"], [35, 13, 18, 13, 18]):
    ws2.column_dimensions[col].width = w

wb.save(output_path)

def build_message(df):
    bank = str(df.iloc[0, 0]).strip()
    period = str(df.iloc[1, 0]).strip()
    owner = str(df.iloc[3, 7]).strip() if pd.notna(df.iloc[3, 7]) else ""

def extract_num(s):
    nums = re.findall(r'\d[\d\s]*\.\d*', str(s).replace('\xa0', ' ').replace(
    for n in nums:
        try:
            return float(n)
        except:
            continue
    return 0

```

```

bal_start = extract_num(str(df.iloc[3, 0]))
bal_end = extract_num(str(df.iloc[3, 1]))
txs, total_debit, total_credit = extract_transactions(df)
debit_by, credit_by = build_summary(txs)
all_names = set(list(debit_by.keys()) + list(credit_by.keys()))

msg = []
msg.append("*" + bank + "*")
msg.append(period)
if owner and owner != "nan":
    msg.append("Egasi: *" + owner + "*")
msg.append("")
msg.append("Boshlanish: `%s` sum" % "{:,.0f}".format(bal_start))
msg.append("Tugash: `%s` sum" % "{:,.0f}".format(bal_end))
msg.append("")
msg.append("Jami chiqim: *%s sum*" % "{:,.0f}".format(total_debit))
msg.append("Jami kirim: *%s sum*" % "{:,.0f}".format(total_credit))
msg.append("")
msg.append("FIRMALAR:")
msg.append("")
for name in sorted(all_names):
    d = debit_by.get(name, {"sum": 0, "count": 0})
    cr = credit_by.get(name, {"sum": 0, "count": 0})
    msg.append("*" + name + "*")
    if d["sum"]:
        msg.append("  Chiqim: `%s` sum (%d ta)" % ("{:,.0f}".format(d["sum"])))
    if cr["sum"]:
        msg.append("  Kirim: `%s` sum (%d ta)" % ("{:,.0f}".format(cr["sum"])))
return "\n".join(msg)

async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text(
        "Salom!\n\n"
        ".xlsb yoki .xlsx bank kochirmasini yuboring.\n"
        "Men xabar va Excel hisobot qaytaraman."
    )

async def handle_file(update: Update, context: ContextTypes.DEFAULT_TYPE):
    doc = update.message.document
    if not doc:
        return
    fname = doc.file_name or ""
    if not (fname.endswith(".xlsb") or fname.endswith(".xlsx")):
        await update.message.reply_text("Iltimos, .xlsb yoki .xlsx fayl yuboring.")
        return
    await update.message.reply_text("Fayl qayta ishlanmoqda...")
    with tempfile.TemporaryDirectory() as tmpdir:

```

```

    fpath = os.path.join(tmpdir, fname)
    file = await doc.get_file()
    await file.download_to_drive(fpath)
    try:
        df = parse_xlsb(fpath)
        msg = build_message(df)
        if len(msg) > 4000:
            msg = msg[:4000] + "\n..."
        await update.message.reply_text(msg, parse_mode="Markdown")
        out_path = os.path.join(tmpdir, "hisobot.xlsx")
        create_excel(df, out_path)
        with open(out_path, "rb") as f:
            await update.message.reply_document(
                document=f,
                filename="bank_hisoboti.xlsx",
                caption="Excel hisoboti tayyor!"
            )
    except Exception as e:
        await update.message.reply_text("Xato: " + str(e))

async def handle_other(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text("Iltimos, .xlsb yoki .xlsx fayl yuboring.")

def main():
    app = ApplicationBuilder().token(TOKEN).build()
    app.add_handler(CommandHandler("start", start))
    app.add_handler(MessageHandler(filters.Document.ALL, handle_file))
    app.add_handler(MessageHandler(filters.TEXT & ~filters.COMMAND, handle_other))
    print("Bot ishlamoqda...")
    app.run_polling()

if __name__ == "__main__":
    main()

```